Meridian

Secure Job Search & Professional Networking Platform

Milestone 2 Report

Authentication, 2FA, Profiles, Resume Encryption, Admin

CSE 345/545 — Foundations of Computer Security

Team Members	Roll Number
Aniket Gupta	2022073
Angadjeet Singh	2022071
Harsh Kumar	2022198

IIIT Delhi

February 27, 2026

Contents

1 Overview

This report covers Milestone 2 of the Meridian platform, due February 27, 2026. The milestone required five deliverables:

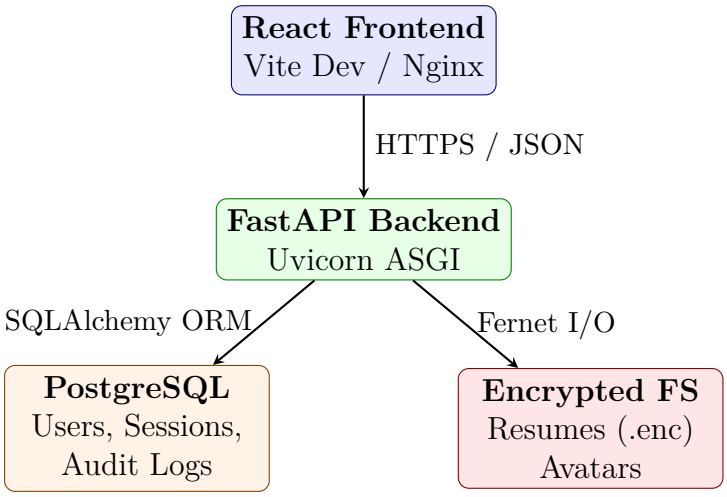
1. Secure user registration and login
2. TOTP-based two-factor authentication (replacing email OTP per instructor guidance)
3. User profile management
4. Secure resume upload with encryption at rest
5. Basic admin dashboard

All five goals are fully implemented and tested. This report details each goal's design, implementation, and security rationale, followed by a summary of the security measures protecting the platform.

1.1 Technology Stack

Layer	Technology	Rationale
Backend	Python / FastAPI	Async, auto-docs, Pydantic validation, dependency injection
Frontend	React 18 / Vite	Component reuse, JSX auto-escaping (XSS defense)
Database	PostgreSQL 16	UUID support, JSONB for audit details, ACID compliance
Deployment	Docker + GitHub Actions	Reproducible builds, automated CI/CD

1.2 Architecture



2 Milestone 2 — Authentication, Profiles, Resumes, Admin

Deadline: February 27, 2026 Status: Completed

This milestone transforms the skeleton from M1 into a functional, security-hardened platform. It delivers five interconnected goals: secure authentication, TOTP-based 2FA, profile management, encrypted resume upload, and an admin dashboard. Each goal builds on the previous — authentication protects profiles, profiles hold resumes, and the admin dashboard oversees everything.

2.1 Goal 1: Secure Authentication

Authentication is the front door to every other feature, so we invested heavily in making it resilient.

2.1.1 Registration

When a user registers, their password goes through **Argon2id** hashing — a memory-hard algorithm designed to resist GPU-based cracking. We configure it with 64 MB memory cost, 3 iterations, and 4 parallel threads. The system also validates password strength (minimum 8 characters, mixed case, digits, and symbols) and rejects common passwords before hashing.

To prevent email enumeration, failed registrations return a generic error regardless of whether the email already exists.

2.1.2 Login

The login flow incorporates several defenses that work together:

1. **CSRF check** — The double-submit cookie pattern verifies the request is genuine.
2. **Lockout check** — After 5 failed attempts, the account locks for 15 minutes.
3. **Timing-safe verification** — If the email doesn't exist, we still compute a dummy Argon2 hash to prevent timing-based enumeration.
4. **2FA branch** — If TOTP is enabled, a short-lived temp token (5 min) is returned instead of session cookies. The user must verify with their authenticator app before gaining access.

On successful login, a database session is created with a device fingerprint (hashed IP + User-Agent) bound to the token. This means a stolen JWT cannot be used from a different device.

2.1.3 Token Architecture

We use a dual-token system stored exclusively in HttpOnly cookies — never in localStorage:

- **Access token** (15 min TTL) — Carries user ID, role, session ID, and device fingerprint. Used for API authorization.
- **Refresh token** (24h TTL) — Scoped to `/api/auth/refresh`. Contains a unique JTI that rotates on every refresh.

If a previously-used JTI appears (suggesting token theft), **all of that user's sessions are immediately revoked** — a defense inspired by OAuth 2.0 refresh token rotation best practices.

2.2 Goal 2: TOTP Two-Factor Authentication

Per instructor guidance, we implement TOTP (RFC 6238) rather than email/SMS OTP. This has practical advantages: it works offline, is compatible with standard authenticator apps (Google Authenticator, Authy), and requires no third-party SMS provider.

2.2.1 Setup

The setup process is designed to prevent accidental lock-outs:

1. The backend generates a random Base32 secret using `pyotp`.
2. The secret is **Fernet-encrypted** before being stored in the database — even a database breach won't expose TOTP seeds.
3. A QR code is generated from the `otpauth://` URI and sent to the frontend as Base64-encoded PNG.
4. The user scans the QR code and must *confirm* by entering a valid code.
5. Only after confirmation is 2FA activated. This prevents the user from enabling 2FA without actually setting up their authenticator.

2.2.2 Replay Prevention

A valid TOTP code is reusable within its 30-second window, which opens a replay attack vector. We close it by recording every verified code in a `used_otps` table with its time step. Before accepting a code, the system checks if it's been used in the current or adjacent windows (to account for clock drift). Old entries are garbage-collected after 5 minutes.

2.3 Goal 3: User Profile Management

With authentication in place, users can build their professional profiles.

2.3.1 Profile Data

The profile supports rich professional information: name, headline, location, bio, education history, work experience, and skills. Education and experience entries support full CRUD operations, while skills use an autocomplete interface backed by a curated list of 150+ professional skills.

2.3.2 Input Sanitization

Every text field passes through a sanitizer that strips HTML tags (via `bleach`), removes JavaScript event handlers, and blocks `javascript:` URIs. This server-side sanitization, combined with React's automatic JSX escaping on the client, creates a two-layer XSS defense.

2.3.3 Avatar Security

Profile pictures go through a validation pipeline that goes beyond simple extension checking:

1. Extension whitelist (`.jpg`, `.jpeg`, `.png`)
2. MIME type verification
3. **Magic byte validation** — the first bytes must match JPEG (`FF D8 FF`) or PNG (`89 50 4E 47`) signatures, preventing disguised file attacks
4. 2 MB size cap
5. UUID-based filename — the original name never reaches the filesystem

2.4 Goal 4: Encrypted Resume Storage

Resumes are the most sensitive data on the platform. A breach that exposes plaintext resumes would compromise names, addresses, phone numbers, and career histories of every user. Our approach ensures that even if an attacker gains filesystem access, the data remains encrypted.

2.4.1 The Upload Pipeline

Uploads go through a multi-stage process before reaching disk:

1. **Validation** — Extension (`.pdf`), MIME type (`application/pdf`), magic bytes (`%PDF-`), size (2 MB limit), and content scan for suspicious embedded JavaScript, `/Launch`, and `/OpenAction` commands.
2. **Encryption** — The validated content is encrypted with **Fernet** (AES-128-CBC + HMAC-SHA256). The encryption key is a 256-bit key stored in an environment variable, never in code.
3. **Storage** — Written as `<uuid>.enc` on disk. The original filename exists only in the database.

2.4.2 The Download Pipeline

On download, the process reverses — but critically, the decrypted content **never touches disk**. It's decrypted in memory and streamed directly to the client over HTTPS. Every download is recorded in the audit log.

2.4.3 Verification

Encrypted files on disk are completely unreadable without the key. Examining a stored resume shows only the Fernet token (starting with `gAAA...`), confirming that no plaintext leaks to the filesystem.

2.5 Goal 5: Admin Dashboard

The admin dashboard provides platform-wide visibility and control. Admins can list all users, change roles, suspend accounts (which immediately revokes all active sessions), and permanently delete users (cascading to resumes, sessions, and OTP records).

All admin actions — especially destructive ones like deletion — are logged in the audit trail with the admin's identity and the target user's details. This creates accountability even for privileged users.

2.5.1 Audit Events

The system logs over 20 distinct event types, including: registration, login success/failure, 2FA setup, account lockout, resume upload/download/deletion, role changes, suspensions, and — importantly — `refresh_token_reuse_detected`, which signals a potential token theft incident.

Each event captures the timestamp, actor ID, client IP, and a JSONB payload with contextual details specific to that event type.

2.6 User Interface

The frontend uses a dark theme with glassmorphism design and smooth animations. Below are screenshots of the key interfaces delivered in Milestone 2.

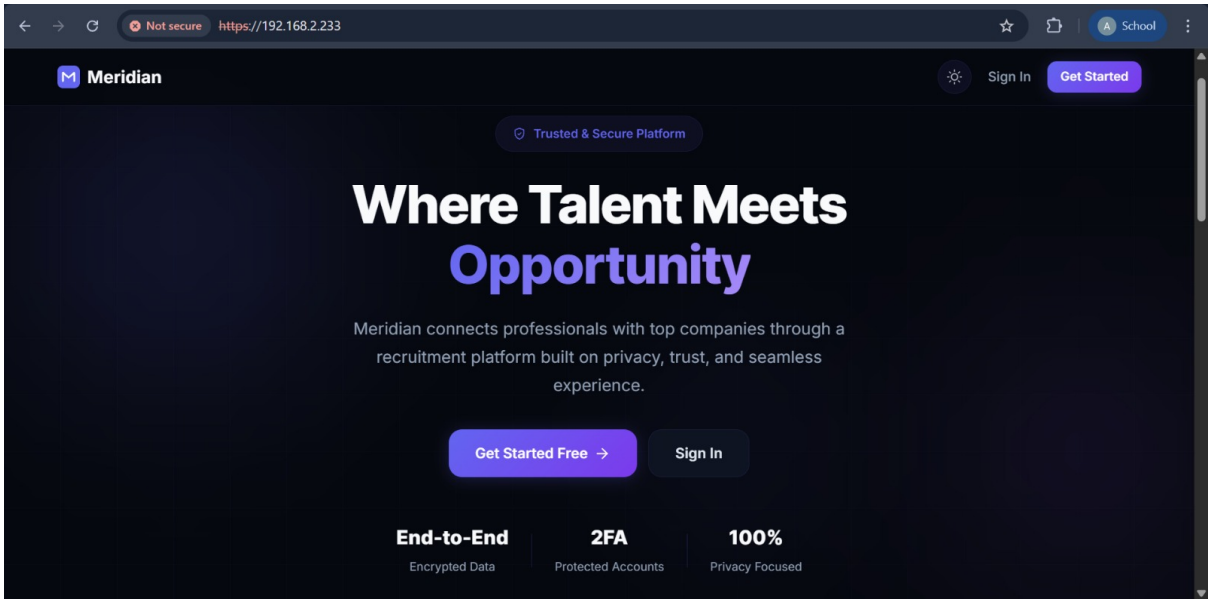


Figure 1: Landing page — hero section with value proposition and security highlights.

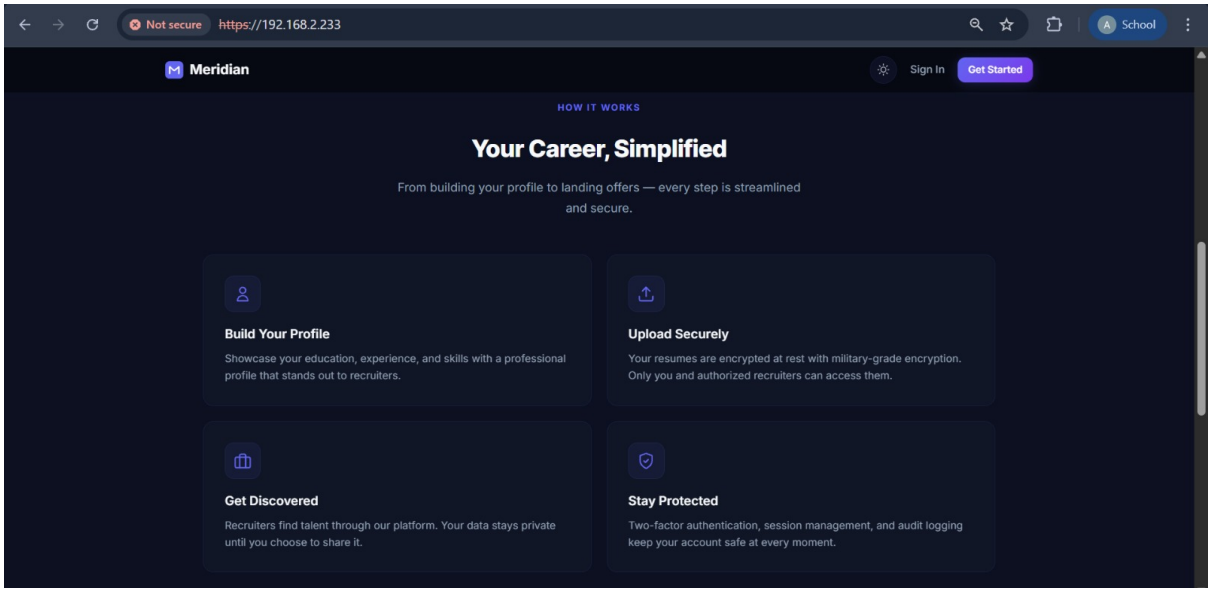
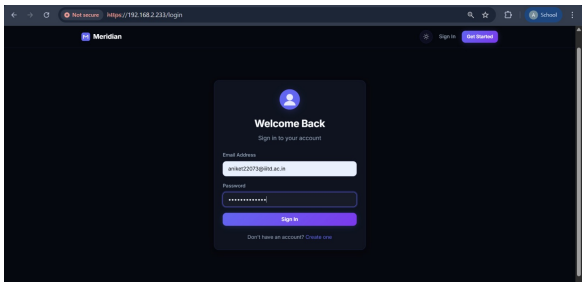
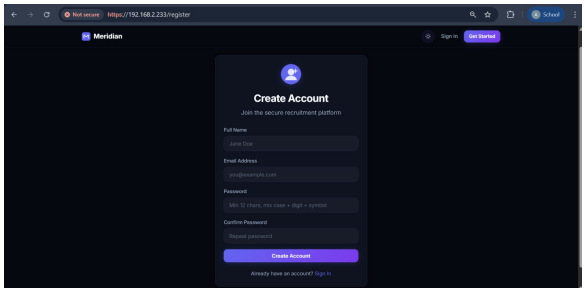


Figure 2: Landing page — feature cards highlighting encryption, 2FA, and audit capabilities.



(a) Login page



(b) Registration page

Figure 3: Authentication pages with gradient avatars and glassmorphism cards.

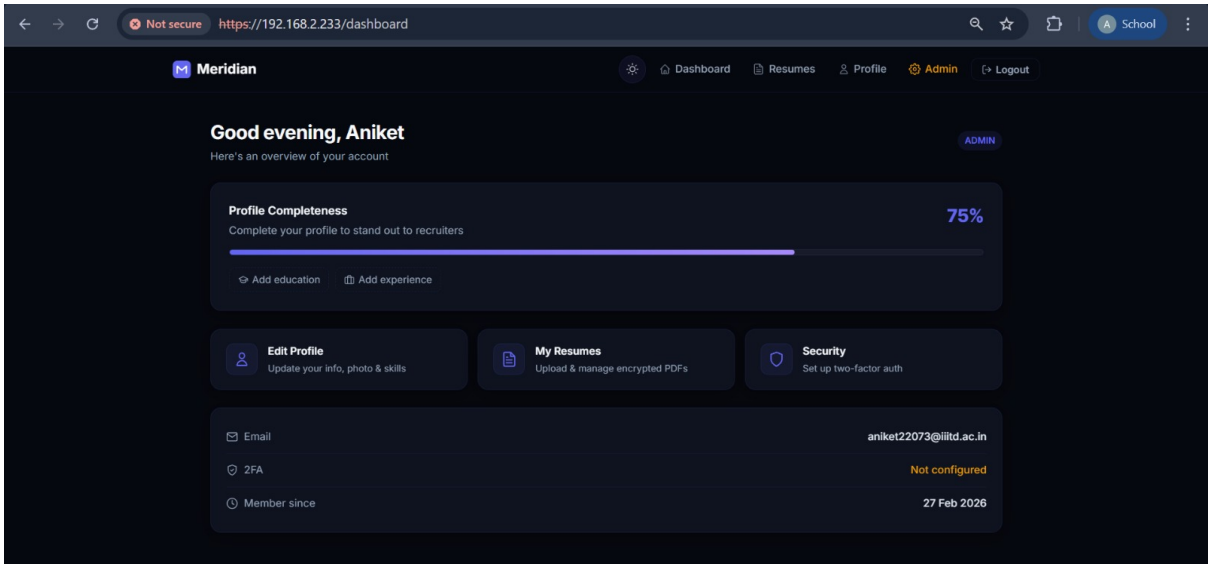


Figure 4: Dashboard — profile completeness indicator, quick actions, and account summary.

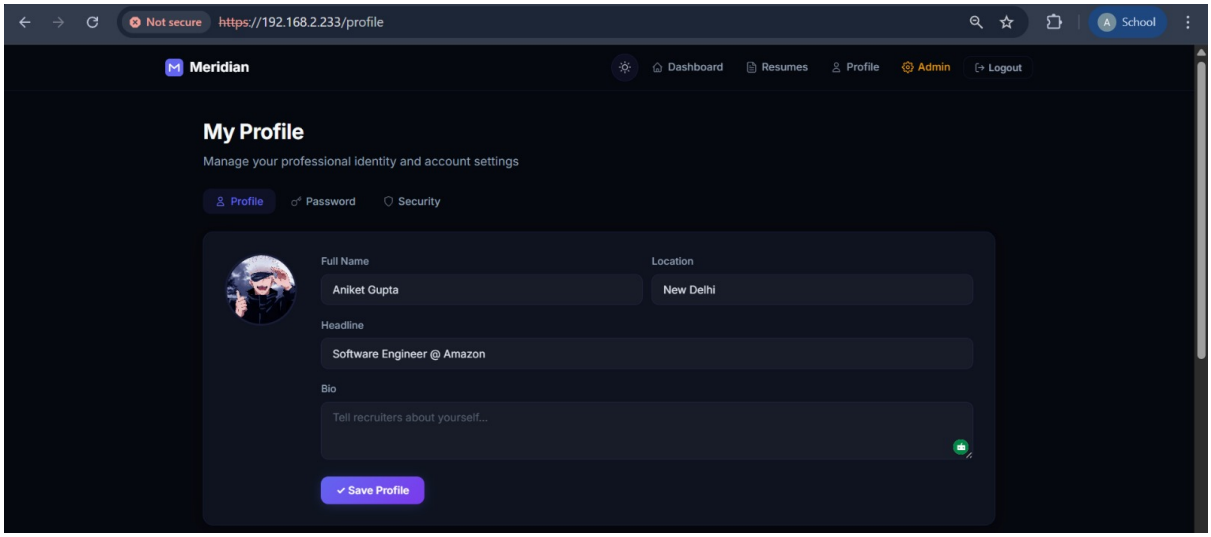


Figure 5: Profile management — education, experience, and skills with inline editing.

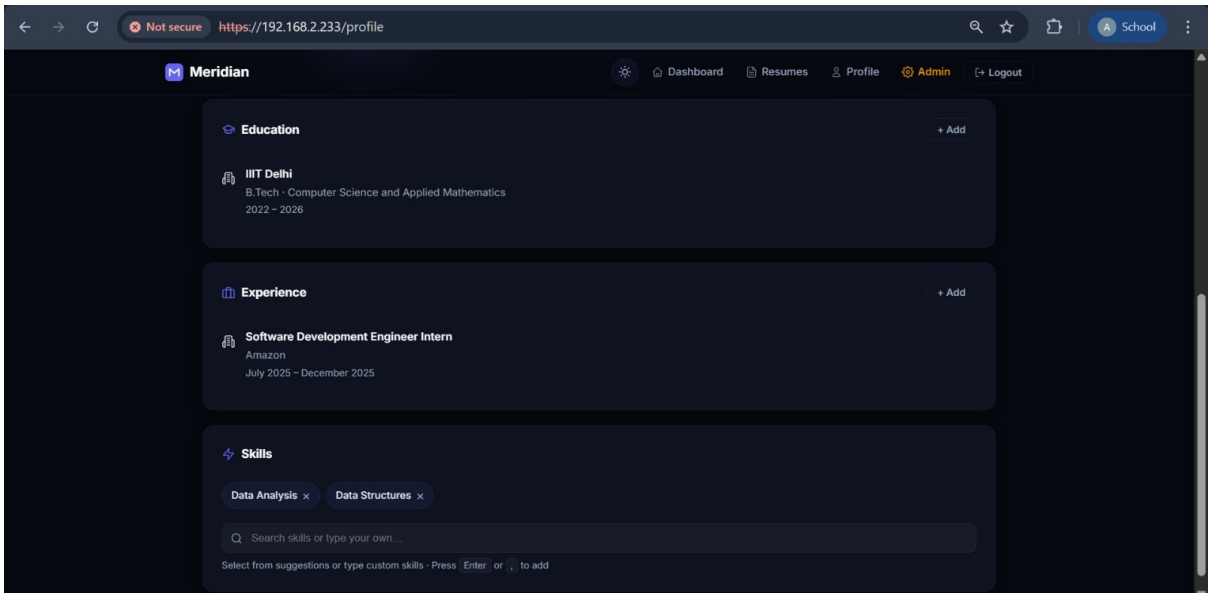


Figure 6: Profile fields — sanitized text inputs with real-time validation.

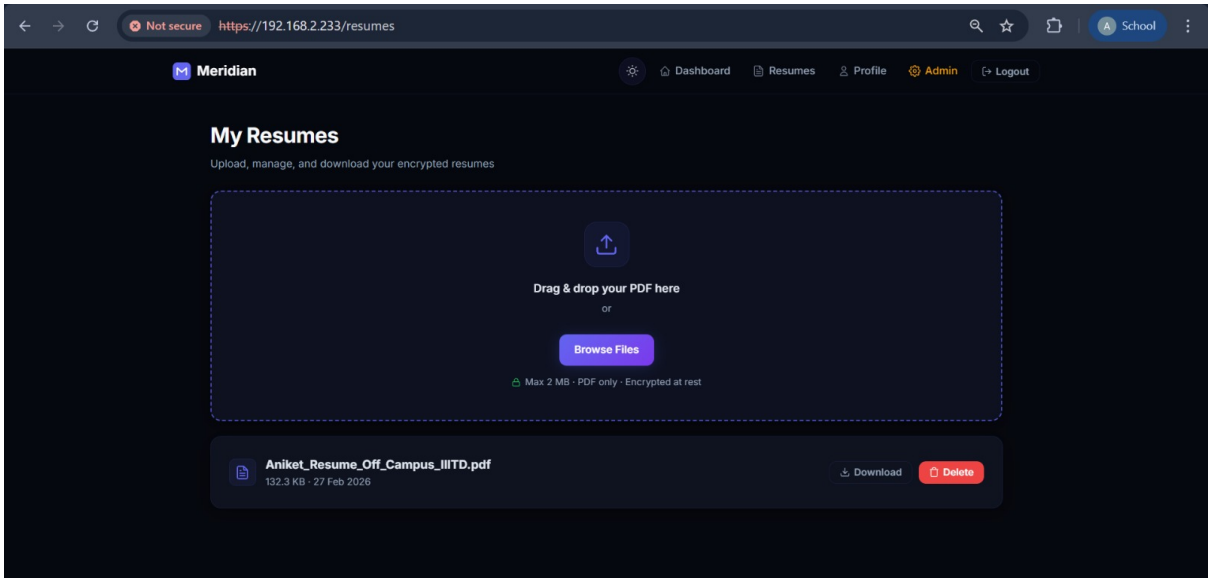


Figure 7: Resume management — encrypted upload, listing, and secure download.

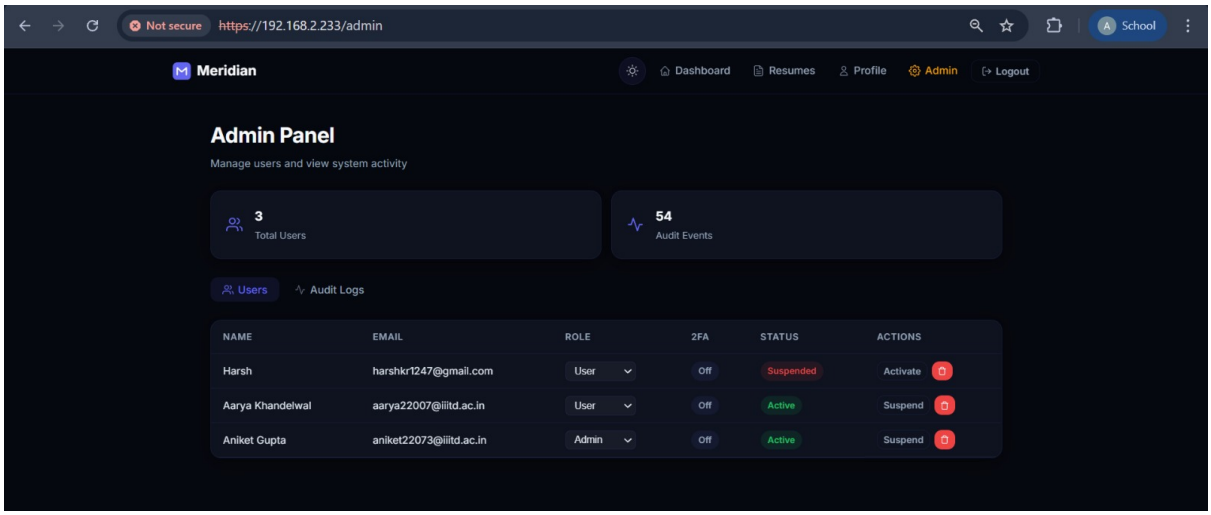


Figure 8: Admin panel — user management with role promotion, suspension, and deletion.

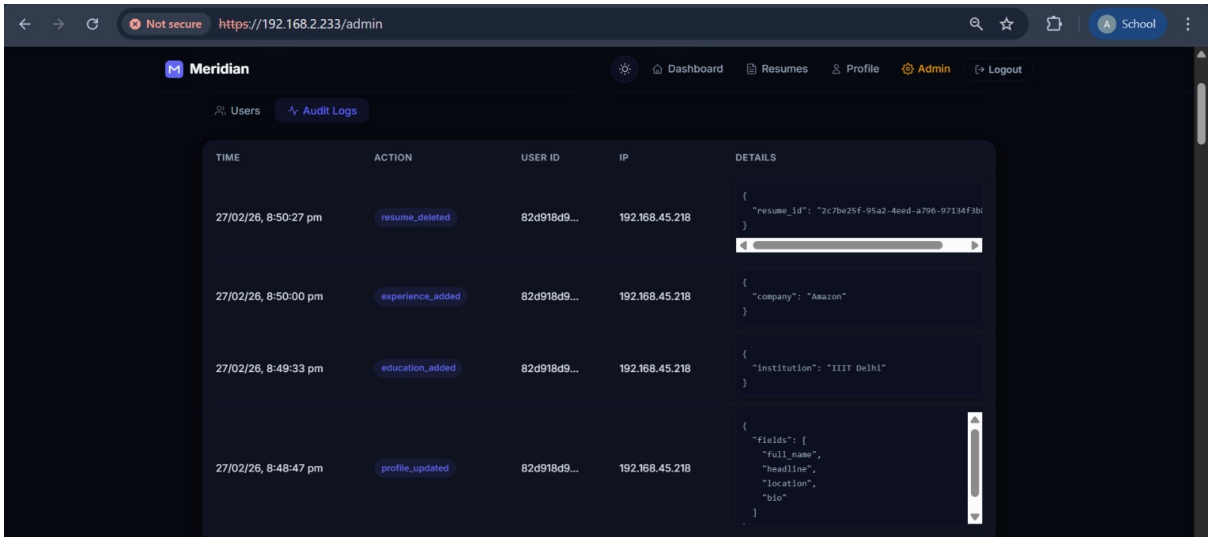


Figure 9: Audit logs — every security event with timestamp, actor, IP, and contextual details.

3 Security Analysis

This section takes a threat-driven look at the platform's security posture. Rather than listing features in isolation, we examine each attack vector and explain how the architecture defends against it.

3.1 Authentication Attacks

Password cracking. Even if the database is compromised, passwords are protected by Argon2id with 64 MB memory cost — making GPU-based brute force prohibitively expensive. Each hash uses a unique 16-byte salt.

Credential stuffing. Automated login attempts from breached credentials are throttled by account lockout (5 attempts triggers a 15-minute lock). The lockout event is logged, alerting admins to potential attacks.

User enumeration. Both registration and login return generic error messages. On login with an unknown email, a dummy hash is computed to make the response time indistinguishable from a valid attempt.

Token theft via XSS. JWTs live in HttpOnly cookies — JavaScript cannot access them, even in a successful XSS scenario. Combined with server-side input sanitization and React's automatic escaping, XSS attack surface is minimal.

Session hijacking. Each token is bound to a device fingerprint (hash of IP + User-Agent). A stolen token used from a different device will be rejected. Additionally, refresh token rotation means a stolen refresh token can only be used once before the entire session graph is invalidated.

3.2 Web Application Attacks

Cross-Site Scripting (XSS) is mitigated at two layers. Server-side, `bleach.clean()` strips all HTML tags, event handlers, and JavaScript URIs from user input before storage. Client-side, React's JSX escaping prevents DOM injection. Security headers (`X-Content-Type-Options: nosniff`) prevent MIME-sniffing attacks.

Cross-Site Request Forgery (CSRF) is prevented via the double-submit cookie pattern. A random token is set as both a cookie and returned in the response body. State-changing requests must include the token in an `X-CSRF-Token` header, which must match the cookie. Since an attacker's site cannot read our cookies (same-origin policy), they cannot forge the header.

SQL Injection is structurally prevented — all queries use SQLAlchemy's ORM with parameterized bindings. No raw SQL strings exist in the codebase. Input validation via Pydantic schemas adds a second layer of defense.

Session Fixation is impossible because sessions are created server-side with fresh UUIDs. The client cannot influence session identifiers. On every login, a new session is created; on logout, the existing session is database-revoked.

3.3 Data Protection

Sensitive data is protected both in transit and at rest:

- **Passwords** — Argon2id one-way hash (irreversible)

- **Resumes** — Fernet encryption on disk (AES-128-CBC + HMAC-SHA256); decrypted only in memory during authorized downloads
- **TOTP secrets** — Fernet-encrypted in the database; decrypted only during verification
- **Transport** — All traffic over HTTPS with TLS

File uploads face a five-layer validation pipeline (extension, MIME, magic bytes, size, content scan for PDFs) before encryption. Files are stored with UUID names to prevent path traversal, and the storage directory sits outside the web root.

3.4 Access Control

The RBAC model is enforced at the API layer, not the UI layer — meaning even if someone bypasses the frontend, the backend rejects unauthorized requests:

Resource	User	Recruiter	Admin
Own profile	Read/Write	Read/Write	Read/Write
Own resumes	CRUD	CRUD	CRUD
Others' resumes	—	Read	Read
User management	—	—	Full
Audit logs	—	—	Read

Table 1: RBAC permission matrix. Enforced via FastAPI `Depends()` injection.

3.5 Security Headers

Every response includes hardened headers that defend against common browser-based attacks:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self';
   ...
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

4 What's Next — Milestone 3

With the authentication and data protection foundation in place, Milestone 3 (March 31) will add:

- Company pages and job postings (recruiter workflow)
- Job search with filters (keywords, location, remote, skills)
- Application tracking (Applied → Reviewed → Interviewed → Offer/Rejected)
- End-to-end encrypted 1:1 messaging between recruiters and candidates
- Expanded audit logging for all new features